Operating Systems Course
Project Handout

CPU Scheduling Comparison Projects

Student-facing formal handout for five approved comparison-project variants

This handout applies to the Operating Systems project. Each team is assigned one comparison project and must implement both algorithms, run them on the same workloads, and produce a technically sound comparison with clear metrics, Gantt charts, validation, and written analysis.

3061220210341B

1. Purpose of this Handout

This document defines the five approved comparison projects for the Operating Systems course project.

In every project, the team must implement two scheduling algorithms, execute them on the same workload, and explain the differences in behavior, performance, fairness, and scheduling trade-offs.

The goal is not only to build a simulator, but also to demonstrate understanding of CPU scheduling concepts through accurate comparison and technical interpretation.

2. Common Requirements for All Five Projects

Every project must include a clear interface, separate Gantt charts for both algorithms, user-defined process input at runtime, input validation, per-process Waiting Time (WT), Turnaround Time (TAT), and Response Time (RT), plus average WT, average TAT, and average RT.

All comparisons must use the same workload for both algorithms. A comparison is not valid if the input dataset differs between the two runs.

At minimum, the simulator must accept Process ID, Arrival Time, and Burst Time. Priority-based projects must also accept Priority Value. Round Robin projects must also accept Time Quantum.

The simulator must safely reject invalid data such as negative arrival times, zero or negative burst times, duplicate process IDs, invalid priority values, invalid quantum values, missing required fields, and non-numeric input in numeric fields.

3. Approved Projects

Code	Project	Algorithms Compared	Main Focus	Extra Input
C1	Priority vs SRTF	Priority + SRTF	Policy vs shortest remaining time	Priority
C2	Round Robin vs SRTF	RR + SRTF	Fairness vs efficiency	Quantum
C3	Round Robin vs Priority	RR + Priority	Fairness vs urgency	Quantum + Priority
C4	SJF vs Priority	SJF + Priority	Shortest job vs urgency	Priority
C5	Round Robin vs SJF	RR + SJF	Time slicing vs shortest job	Quantum

3061220210341 3

SJF vs Priority Comparison Project

Project at a glance: Algorithms compared: **Shortest Job First (SJF)** and **Priority Scheduling**. Main focus: shortest available job versus urgency or importance. **Special input**: Priority value.

Project Objective

In this project, your team will implement and compare Shortest Job First (SJF) and Priority Scheduling.

The purpose is to compare a scheduler that prefers the shortest available burst time with a scheduler that prefers the highest-priority process according to a defined rule.

Your team is expected to study how job length versus urgency affects execution order, average metrics, fairness, and starvation risk.

Required Functionality

- Accept a dynamic number of processes and all required process data at runtime.
- Validate all input safely before simulation begins.
- Implement SJF correctly; unless otherwise instructed, SJF should be non-preemptive.
- Implement Priority Scheduling correctly, with a clearly stated priority rule and documented tie handling.
- Display separate Gantt charts and separate metrics tables for both algorithms.
- Calculate WT, TAT, RT, average WT, average TAT, and average RT.

Required Comparison Focus

- How SJF behaves when a short job has low priority.
- How Priority behaves when a long job has very high priority.
- Whether SJF improves average waiting or turnaround time.
- Whether Priority improves service for urgent processes.
- Whether either algorithm causes unfair delay.

Required Interface Sections

- Input Panel
- Process Table
- Priority Input Area
- Gantt Chart for SJF
- Gantt Chart for Priority
- Results Table for SJF
- Results Table for Priority
- Comparison Summary Section
- Final Conclusion Area

Required Test Scenarios

Scenario A: Basic mixed workload

- Use a normal workload with different arrival times and burst times.

Scenario B: Conflict between burst time and priority

- Include a short-burst low-priority process and a long-burst high-priority process to reveal a meaningful difference.

Scenario C: Fairness or starvation-sensitive case

- Prepare a workload where one process may wait much longer under one of the algorithms.

3001220210341B

Scenario D: Validation case

- Include at least one invalid input example and show the validation behavior.

Required Analysis Questions

- Which algorithm gave lower average waiting time?
- Which algorithm gave lower average turnaround time?
- Did SJF favor short jobs more strongly?
- Did Priority Scheduling favor urgent processes more strongly?
- Was any starvation or unfair delay observed?
- Which algorithm would you recommend for the tested workload, and why?

Required Conclusion

- State which algorithm performed better on the selected datasets.
- State which metric each algorithm handled better.
- Explain the trade-off between efficiency and urgency.
- State which algorithm appeared fairer in practice.

3061220210341 3